DATA STRUCTURES & ALGORITHMS PRACTICAL MANUAL

Strict Template: Problem Statement | Algorithm | Input & Expected Output | Source Code | Actual Output

Practical 9: Searching Algorithms

Problem Statement

> Implement Linear Search, binary search and interpolation search.

Algorithm

> 1. Linear Search:
> - Sequentially check each element of the array from index 0 to n-1 against the search key.
> - If arr[i] == key, return index i.
> - If end of array is reached without a match, return -1.
> - Time Complexity: Best: O(1), Worst/Average: O(n) | Space Complexity: O(1)
>
> 2. Binary Search (Requires Sorted Array):
> - Set low = 0 and high = n - 1.
> - While low <= high:
> a. Calculate mid = low + (high - low) / 2.
> b. If arr[mid] == key, return mid.
> c. Else if arr[mid] < key, set low = mid + 1.
> d. Else set high = mid - 1.
> - If key is not found, return -1.
> - Time Complexity: Best: O(1), Worst/Average: O(log n) | Space Complexity: O(1)
>
> 3. Interpolation Search (Requires Sorted, Uniformly Distributed Array):
> - Set low = 0 and high = n - 1.
> - While low <= high and key >= arr[low] and key <= arr[high]:
> a. If low == high: return low if arr[low] == key else -1.
> b. Estimate probe index:
> pos = low + [((high - low) / (arr[high] - arr[low])) * (key - arr[low])]
> c. If arr[pos] == key, return pos.
> d. Else if arr[pos] < key, set low = pos + 1.
> e. Else set high = pos - 1.
> - Return -1 if not found.
> - Time Complexity: Best: O(1), Average: O(log(log n)), Worst: O(n) | Space Complexity: O(1)

Input and Expected Output Test Case

> Input:

- Array: [10, 20, 30, 40, 50, 60, 70]
- Search Key: 40

Expected Output:
- Linear Search: Found at index 3
- Binary Search: Found at index 3
- Interpolation Search: Found at index 3

Source Code

```
#include <iostream>
#include <vector>
using namespace std;

int linearSearch(const vector<int>& arr, int key) {
    for (int i = 0; i < (int)arr.size(); i++) {
        if (arr[i] == key) return i;
    }
    return -1;
}

int binarySearch(const vector<int>& arr, int key) {
    int low = 0, high = (int)arr.size() - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == key) return mid;
        if (arr[mid] < key) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int interpolationSearch(const vector<int>& arr, int key) {
    int low = 0, high = (int)arr.size() - 1;
    while (low <= high && key >= arr[low] && key <= arr[high]) {
        if (low == high) {
            return (arr[low] == key) ? low : -1;
        }
        int pos = low + (((double)(high - low) / (arr[high] - arr[low])) * (key - arr[low]));
        if (arr[pos] == key) return pos;
        if (arr[pos] < key) low = pos + 1;
        else high = pos - 1;
    }
    return -1;
}

int main() {
    vector<int> arr = {10, 20, 30, 40, 50, 60, 70};
    int key = 40;

    cout << "Linear Search: Found at index " << linearSearch(arr, key) << "\n";
```

```
    cout << "Binary Search: Found at index " << binarySearch(arr, key) << "\n";
    cout << "Interpolation Search: Found at index " << interpolationSearch(arr, key) << "\n";

    return 0;
}
```

Actual Output

```
Linear Search: Found at index 3
Binary Search: Found at index 3
Interpolation Search: Found at index 3
```

Practical 10: Sorting Algorithms

Problem Statement

Implement Bubble sort, selection sort, Insertion sort, quick sort, merge sort.

Algorithm

1. Bubble Sort:
   - Repeatedly iterate through the list, comparing adjacent elements and swapping them if arr[j] > arr[j+1].
   - An optimized flag tracks if swaps occurred; if none occurred, array is sorted.
   - Time Complexity: Best: O(n), Average/Worst: O(n^2) | Space Complexity: O(1)

2. Selection Sort:
   - Divide list into sorted and unsorted subarrays.
   - Find minimum element in unsorted portion and swap it with first element of unsorted portion.
   - Time Complexity: Best/Average/Worst: O(n^2) | Space Complexity: O(1)

3. Insertion Sort:
   - Iterate through array; pick current element key and insert it into sorted position among elements to its left.
   - Time Complexity: Best: O(n), Average/Worst: O(n^2) | Space Complexity: O(1)

4. Quick Sort:
   - Pick a pivot element (Lomuto partition scheme uses last element).
   - Rearrange elements such that elements < pivot are placed left, and elements >= pivot are placed right.
   - Recursively apply quick sort on left and right partitions.
   - Time Complexity: Best/Average: O(n log n), Worst: O(n^2) | Space Complexity: O(log n) stack space

5. Merge Sort:
   - Divide array into two halves at mid = (l + r) / 2.
   - Recursively sort each half, then merge two sorted halves into a single sorted array.
   - Time Complexity: Best/Average/Worst: O(n log n) | Space Complexity: O(n) auxiliary space

Input and Expected Output Test Case

Input:
- Unsorted Array: [64, 25, 12, 22, 11]

Expected Output:
- Bubble Sort: 11 12 22 25 64
- Selection Sort: 11 12 22 25 64
- Insertion Sort: 11 12 22 25 64
- Quick Sort: 11 12 22 25 64
- Merge Sort: 11 12 22 25 64

Source Code

```
#include <iostream>
#include <vector>
using namespace std;

void printArray(const vector<int>& arr) {
    for (int x : arr) cout << x << " ";
    cout << "\n";
}

void bubbleSort(vector<int> arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
    cout << "Bubble Sort: ";
    printArray(arr);
}

void selectionSort(vector<int> arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx]) minIdx = j;
        }
        swap(arr[i], arr[minIdx]);
    }
    cout << "Selection Sort: ";
    printArray(arr);
}

void insertionSort(vector<int> arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
    cout << "Insertion Sort: ";
    printArray(arr);
```

```
}

int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSortHelper(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSortHelper(arr, low, pi - 1);
        quickSortHelper(arr, pi + 1, high);
    }
}

void quickSort(vector<int> arr) {
    quickSortHelper(arr, 0, arr.size() - 1);
    cout << "Quick Sort: ";
    printArray(arr);
}

void merge(vector<int>& arr, int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}

void mergeSortHelper(vector<int>& arr, int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSortHelper(arr, l, m);
        mergeSortHelper(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

void mergeSort(vector<int> arr) {
```

```
    mergeSortHelper(arr, 0, arr.size() - 1);
    cout << "Merge Sort: ";
    printArray(arr);
}

int main() {
    vector<int> arr = {64, 25, 12, 22, 11};
    bubbleSort(arr);
    selectionSort(arr);
    insertionSort(arr);
    quickSort(arr);
    mergeSort(arr);
    return 0;
}
```

Actual Output

```
Bubble Sort: 11 12 22 25 64
Selection Sort: 11 12 22 25 64
Insertion Sort: 11 12 22 25 64
Quick Sort: 11 12 22 25 64
Merge Sort: 11 12 22 25 64
```

Practical 11: Binary Search Tree Operations

Problem Statement

> Implement Binary search Tree and its operations (creation, insertion, deletion).

Algorithm

> 1. Creation & Node Structure:
> - Define a Node containing key, left pointer, and right pointer. Initially root = NULL.
>
> 2. Insertion Operation:
> - If root is NULL, allocate new node with given key and return it.
> - If key < root->key, recursively insert in root->left.
> - If key > root->key, recursively insert in root->right.
> - Time Complexity: Best/Average: O(log n), Worst: O(n) | Space Complexity: O(h)
>
> 3. Deletion Operation:
> - Search for the node containing key.
> - Case 1 (Leaf node): Free node and return NULL.
> - Case 2 (One child): Replace node with its non-empty child and delete node.
> - Case 3 (Two children): Find inorder successor (minimum value in right subtree), copy its value to current node, and recursively delete inorder successor from right subtree.
> - Time Complexity: Best/Average: O(log n), Worst: O(n) | Space Complexity: O(h)

Input and Expected Output Test Case

> Input Operations:
> - Insert sequence: 50, 30, 20, 40, 70, 60, 80
> - Delete leaf node: 20
> - Delete node with children: 30
>
> Expected Output:
> - Initial BST (Inorder): 20 30 40 50 60 70 80
> - After Deleting 20: 30 40 50 60 70 80
> - After Deleting 30: 40 50 60 70 80

Source Code

```
#include <iostream>
using namespace std;

struct Node {
    int key;
```

```
    Node* left;
    Node* right;
    Node(int val) : key(val), left(nullptr), right(nullptr) {}
};

Node* insert(Node* root, int key) {
    if (!root) return new Node(key);
    if (key < root->key) root->left = insert(root->left, key);
    else if (key > root->key) root->right = insert(root->right, key);
    return root;
}

Node* minValueNode(Node* node) {
    Node* current = node;
    while (current && current->left) current = current->left;
    return current;
}

Node* deleteNode(Node* root, int key) {
    if (!root) return root;

    if (key < root->key) {
        root->left = deleteNode(root->left, key);
    } else if (key > root->key) {
        root->right = deleteNode(root->right, key);
    } else {
        if (!root->left) {
            Node* temp = root->right;
            delete root;
            return temp;
        } else if (!root->right) {
            Node* temp = root->left;
            delete root;
            return temp;
        }
        Node* temp = minValueNode(root->right);
        root->key = temp->key;
        root->right = deleteNode(root->right, temp->key);
    }
    return root;
}

void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->key << " ";
    inorder(root->right);
}

int main() {
    Node* root = nullptr;
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 20);
```

```
    root = insert(root, 40);
    root = insert(root, 70);
    root = insert(root, 60);
    root = insert(root, 80);

    cout << "Initial BST (Inorder): ";
    inorder(root);
    cout << "\n";

    root = deleteNode(root, 20);
    cout << "After Deleting 20: ";
    inorder(root);
    cout << "\n";

    root = deleteNode(root, 30);
    cout << "After Deleting 30: ";
    inorder(root);
    cout << "\n";

    return 0;
}
```

Actual Output

```
Initial BST (Inorder): 20 30 40 50 60 70 80
After Deleting 20: 30 40 50 60 70 80
After Deleting 30: 40 50 60 70 80
```

Practical 12: Binary Search Tree Traversals

Problem Statement

> Implement Traversals Preorder, Inorder, Postorder on BST.

Algorithm

> 1. Preorder Traversal (Root -> Left -> Right):
> - Visit current root node and process/print its key.
> - Recursively traverse left subtree: preorder(root->left).
> - Recursively traverse right subtree: preorder(root->right).
> - Time Complexity: O(n) | Space Complexity: O(h) recursion stack
>
> 2. Inorder Traversal (Left -> Root -> Right):
> - Recursively traverse left subtree: inorder(root->left).
> - Visit current root node and process/print its key.
> - Recursively traverse right subtree: inorder(root->right).
> - Note: Produces keys in ascending sorted order for any valid BST.
> - Time Complexity: O(n) | Space Complexity: O(h) recursion stack
>
> 3. Postorder Traversal (Left -> Right -> Root):
> - Recursively traverse left subtree: postorder(root->left).
> - Recursively traverse right subtree: postorder(root->right).
> - Visit current root node and process/print its key.
> - Time Complexity: O(n) | Space Complexity: O(h) recursion stack

Input and Expected Output Test Case

> Input:
> - BST Nodes Inserted: [50, 30, 20, 40, 70, 60, 80]
>
> Expected Output:
> - Preorder Traversal: 50 30 20 40 70 60 80
> - Inorder Traversal: 20 30 40 50 60 70 80
> - Postorder Traversal: 20 40 30 60 80 70 50

Source Code

```
#include <iostream>
using namespace std;

struct Node {
    int key;
```

```
    Node* left;
    Node* right;
    Node(int val) : key(val), left(nullptr), right(nullptr) {}
};

Node* insert(Node* root, int key) {
    if (!root) return new Node(key);
    if (key < root->key) root->left = insert(root->left, key);
    else if (key > root->key) root->right = insert(root->right, key);
    return root;
}

void preorder(Node* root) {
    if (!root) return;
    cout << root->key << " ";
    preorder(root->left);
    preorder(root->right);
}

void inorder(Node* root) {
    if (!root) return;
    inorder(root->left);
    cout << root->key << " ";
    inorder(root->right);
}

void postorder(Node* root) {
    if (!root) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->key << " ";
}

int main() {
    Node* root = nullptr;
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 70);
    root = insert(root, 60);
    root = insert(root, 80);

    cout << "Preorder Traversal: ";
    preorder(root);
    cout << "\n";

    cout << "Inorder Traversal: ";
    inorder(root);
    cout << "\n";

    cout << "Postorder Traversal: ";
    postorder(root);
    cout << "\n";
```

```
    return 0;
}
```

Actual Output

```
Preorder Traversal: 50 30 20 40 70 60 80
Inorder Traversal: 20 30 40 50 60 70 80
Postorder Traversal: 20 40 30 60 80 70 50
```

Practical 13: Graph Representations and Traversals

Problem Statement

Implement Graphs and represent using adjacency list and adjacency matrix and implement basic operations with traversals (BFS and DFS).

Algorithm

1. Graph Representation:
   - Adjacency Matrix: 2D vector matrix[V][V] where matrix[u][v] = 1 represents a directed edge u -> v.
     Space Complexity: O(V^2)
   - Adjacency List: Array of vector lists adjList[V] where adjList[u] contains all directed destination vertices.
     Space Complexity: O(V + E)

2. Breadth First Search (BFS):
   - Initialize visited boolean vector of size V to false.
   - Use a FIFO queue; mark start vertex visited and push to queue.
   - While queue is non-empty:
     a. Pop current front node curr and print it.
     b. Traverse all adjacent neighbors of curr; if unvisited, mark visited and enqueue.
   - Time Complexity: O(V + E) | Space Complexity: O(V)

3. Depth First Search (DFS):
   - Initialize visited boolean vector of size V to false.
   - Helper function dfsHelper(v):
     a. Mark v as visited and print v.
     b. Recursively call dfsHelper for each unvisited neighbor of v.
   - Time Complexity: O(V + E) | Space Complexity: O(V) auxiliary recursion stack

Input and Expected Output Test Case

Input:
- Graph with 4 vertices: 0, 1, 2, 3
- Directed Edges: (0, 1), (0, 2), (1, 2), (2, 0), (2, 3), (3, 3)
- Traversal Starting Vertex: 2

Expected Output:
- Adjacency Matrix: 4x4 matrix representing edge connectivity
- Adjacency List: Representation of outgoing edges per vertex
- BFS Traversal (starting from 2): 2 0 3 1
- DFS Traversal (starting from 2): 2 0 1 3

Source Code

```
#include <iostream>
#include <vector>
#include <queue>
using namespace std;

class Graph {
    int V;
    vector<vector<int>> adjMatrix;
    vector<vector<int>> adjList;

    void dfsHelper(int v, vector<bool>& visited) {
        visited[v] = true;
        cout << v << " ";
        for (int neighbor : adjList[v]) {
            if (!visited[neighbor]) {
                dfsHelper(neighbor, visited);
            }
        }
    }

public:
    Graph(int vertices) : V(vertices) {
        adjMatrix.assign(V, vector<int>(V, 0));
        adjList.resize(V);
    }

    void addEdge(int u, int v) {
        adjMatrix[u][v] = 1;
        adjList[u].push_back(v);
    }

    void displayMatrix() {
        cout << "Adjacency Matrix:\n";
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                cout << adjMatrix[i][j] << " ";
            }
            cout << "\n";
        }
    }

    void displayList() {
        cout << "Adjacency List:\n";
        for (int i = 0; i < V; i++) {
            cout << i << " -> ";
            for (int neighbor : adjList[i]) {
                cout << neighbor << " ";
            }
            cout << "\n";
        }
    }

    void bfs(int start) {
```

```
        vector<bool> visited(V, false);
        queue<int> q;

        visited[start] = true;
        q.push(start);

        cout << "BFS Traversal (starting from " << start << "): ";
        while (!q.empty()) {
            int curr = q.front();
            q.pop();
            cout << curr << " ";

            for (int neighbor : adjList[curr]) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.push(neighbor);
                }
            }
        }
        cout << "\n";
    }

    void dfs(int start) {
        vector<bool> visited(V, false);
        cout << "DFS Traversal (starting from " << start << "): ";
        dfsHelper(start, visited);
        cout << "\n";
    }
};

int main() {
    Graph g(4);

    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(1, 2);
    g.addEdge(2, 0);
    g.addEdge(2, 3);
    g.addEdge(3, 3);

    g.displayMatrix();
    cout << "\n";
    g.displayList();
    cout << "\n";

    g.bfs(2);
    g.dfs(2);

    return 0;
}
```

Actual Output

```
Adjacency Matrix:
0 1 1 0
0 0 1 0
1 0 0 1
0 0 0 1

Adjacency List:
0 -> 1 2
1 -> 2
2 -> 0 3
3 -> 3

BFS Traversal (starting from 2): 2 0 3 1
DFS Traversal (starting from 2): 2 0 1 3
```